CSL599: LAB 4
GROUP CHAT APPLICATION

TEAM MEMBERS:
1. AGASTYA NATH 12340140
2. GALABA VAMSI 12340770
3. PARITOSH LAHRE 12341550
4. Y. RAHUL DEV REDDY 12342390

CLIENT URL: https://10.1.75.51:6266/

GITHUB REPO:
https://github.com/SandstromPL/real-time-Group-Chat-Application.git

TECH STACK:
- Python
- Websockets
- React
- Vite with ESLint
- SQLite
- Cryptography
- AES-GCM, RSA-PSS, HTTPS, WSS

PROJECT STRUCTURE:

```
chat-app/
├── backend
│   └── server.py
├── chat-frontend
│   ├── eslint.config.js
│   ├── index.html
│   ├── package.json
│   ├── public
│   │   ├── favicon.svg
│   │   └── icons.svg
│   ├── README.md
│   ├── src
│   │   ├── App.css
│   │   ├── App.jsx
│   │   ├── App1.jsx
│   │   ├── assets
```

```
│   │   │   ├── hero.png
│   │   │   ├── react.svg
│   │   │   └── vite.svg
│   │   ├── index.css
│   │   └── main.jsx
│   └── vite.config.js
└── README.md
```

server.py

```python
import asyncio
import ssl
import base64
import os
import json
import sqlite3
from datetime import datetime, timezone
from pathlib import Path

import websockets
from websockets.exceptions import ConnectionClosed

from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.asymmetric import padding

HOST = "0.0.0.0"
PORT = 5000

DB_PATH = Path(__file__).with_name("chat.db")
KEY_PATH = Path(__file__).with_name("encryption.key")

# Number of most recent messages sent to a user when they join.
HISTORY_LIMIT = 50

# Maps each WebSocket connection to its username.
users = {}
# username -> public signing key
public_keys = {}

ssl_context = ssl.SSLContext(
```

```
    ssl.PROTOCOL_TLS_SERVER
)

ssl_context.load_cert_chain(
    "/home/student/chat-ssl/cert.pem",
    "/home/student/chat-ssl/key.pem",
)

# ---------------------------------------------------------
# Encryption key
# ---------------------------------------------------------

def load_encryption_key():
    """Load the persistent AES-256 key, creating it if necessary."""

    if KEY_PATH.exists():
        return KEY_PATH.read_bytes()

    key = AESGCM.generate_key(bit_length=256)
    KEY_PATH.write_bytes(key)

    return key


ENCRYPTION_KEY = load_encryption_key()
aes = AESGCM(ENCRYPTION_KEY)

def init_db():
    """Create the database and messages table if they do not exist."""

    with sqlite3.connect(DB_PATH) as connection:

        connection.execute(
            """
            CREATE TABLE IF NOT EXISTS users (
                username TEXT PRIMARY KEY,
                public_key TEXT NOT NULL
            )
            """
        )

        connection.execute(
            """
```

```
            CREATE TABLE IF NOT EXISTS messages (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT NOT NULL,
                public_key TEXT NOT NULL,
                ciphertext BLOB NOT NULL,
                nonce BLOB NOT NULL,
                signature BLOB NOT NULL,
                timestamp TEXT NOT NULL
            )
            """
        )


def store_message(username, public_key, ciphertext, nonce, signature,
timestamp):
    """Save a chat message and return its timestamp."""
    #timestamp = datetime.now(timezone.utc).isoformat()

    with sqlite3.connect(DB_PATH) as connection:
        connection.execute(
            "INSERT INTO messages (username, public_key, ciphertext,
nonce, signature, timestamp) VALUES (?, ?, ?, ?, ?, ?)",
            (username, public_key, ciphertext, nonce, signature,
timestamp),
        )



def load_history(limit=HISTORY_LIMIT):
    """Return the most recent messages, oldest first."""
    with sqlite3.connect(DB_PATH) as connection:
        rows = connection.execute(
            """
            SELECT username, public_key, ciphertext, nonce, signature,
timestamp
            FROM messages
            ORDER BY id DESC
            LIMIT ?
            """,
            (limit,),
        ).fetchall()

    return [
```

```
            {"username": username, "public_key": public_key,
"ciphertext": base64.b64encode(ciphertext).decode(), "nonce":
base64.b64encode(nonce).decode(), "signature":
base64.b64encode(signature).decode(), "timestamp": timestamp}
        for username, public_key, ciphertext, nonce, signature,
timestamp in reversed(rows)
    ]

def save_public_key(username, public_key):
    """Store a user's public signing key."""

    with sqlite3.connect(DB_PATH) as connection:
        connection.execute(
            """
            INSERT OR REPLACE INTO users
            (username, public_key)
            VALUES (?, ?)
            """,
            (username, public_key),
        )

def load_public_key(username):
    """Load a user's public signing key."""

    with sqlite3.connect(DB_PATH) as connection:
        row = connection.execute(
            """
            SELECT public_key
            FROM users
            WHERE username = ?
            """,
            (username,),
        ).fetchone()

    if row is None:
        return None

    return row[0]

# ---------------------------------------------------------
# AES-GCM
# ---------------------------------------------------------
```

```
def encrypt_message(message):
    """Encrypt plaintext using AES-GCM."""

    nonce = os.urandom(12)

    ciphertext = aes.encrypt(
        nonce,
        message.encode(),
        None,
    )

    return ciphertext, nonce

def decrypt_message(ciphertext, nonce):
    """Decrypt an AES-GCM encrypted message."""

    plaintext = aes.decrypt(
        nonce,
        ciphertext,
        None,
    )

    return plaintext.decode()

# ------------------------------------------------------------
# Digital signatures
# ------------------------------------------------------------

def verify_signature(public_key_b64, message, signature):
    """Verify a message using the sender's RSA-PSS public key."""

    if public_key_b64 is None:
        return False

    try:
        public_key_bytes = base64.b64decode(public_key_b64)

        public_key = serialization.load_der_public_key(
            public_key_bytes
        )

        public_key.verify(
            signature,
```

```
            message.encode(),
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=32,
            ),
            hashes.SHA256(),
        )

        return True

    except Exception as error:
        print("Signature verification failed:", repr(error))
        return False

# ------------------------------------------------------------
# Broadcasting
# ------------------------------------------------------------

async def broadcast(payload):
    """Send a JSON payload to all currently connected clients."""
    if not users:
        return

    message = json.dumps(payload)

    clients = list(users.keys())

    results = await asyncio.gather(
        *(client.send(message) for client in clients),
        return_exceptions=True,
    )

    # Remove clients whose connection failed while broadcasting.
    for client, result in zip(clients, results):
        if isinstance(result, Exception):
            users.pop(client, None)

# ------------------------------------------------------------
# User registration
# ------------------------------------------------------------

async def register_user(websocket):
    """Receive and validate a username from a new client."""
```

```
    try:
        raw_message = await websocket.recv()
    except ConnectionClosed:
        return None

    try:
        data = json.loads(raw_message)
    except json.JSONDecodeError:
        await websocket.send(json.dumps({
            "type": "error",
            "message": "Invalid registration message.",
        }))
        return None

    if data.get("type") != "register":
        await websocket.send(json.dumps({
            "type": "error",
            "message": "First message must be registration.",
        }))
        return None

    username = data.get("username", "").strip()
    public_key = data.get("public_key")

    if not username:
        await websocket.send(json.dumps({
            "type": "error",
            "message": "Username cannot be empty.",
        }))
        return None

    if len(username) > 20:
        await websocket.send(json.dumps({
            "type": "error",
            "message": "Username must be 20 characters or fewer.",
        }))
        return None

    if not public_key:
        await websocket.send(json.dumps({
            "type": "error",
            "message": "Public key is required.",
        }))
```

```
        return None

    # Usernames are considered unique case-insensitively.
    existing_names = {
        name.lower()
        for name in users.values()
    }

    if username.lower() in existing_names:
        await websocket.send(json.dumps({
            "type": "error",
            "message": "Username already taken.",
        }))
        return None

    # Store public key in memory and database.
    public_keys[username] = public_key
    await asyncio.to_thread(
        save_public_key,
        username,
        public_key,
    )

    return username


async def unregister_user(websocket):
    """Remove a client from the connected users."""
    return users.pop(websocket, None)


# ---------------------------------------------------------
# Client handler
# ---------------------------------------------------------

async def handle_client(websocket):
    """Handle the complete session of one connected client."""

    username = await register_user(websocket)

    if username is None:
        await websocket.close()
        return
```

```
# Send recent chat history to the new user only.
# This must happen BEFORE adding the client to `users`, otherwise a
# broadcast could reach it before it has received the history.
history = await asyncio.to_thread(load_history)

decrypted_history = []

for item in history:

    try:
        ciphertext = base64.b64decode(item["ciphertext"])
        nonce = base64.b64decode(item["nonce"])
        signature = base64.b64decode(item["signature"])

        message = decrypt_message(
            ciphertext,
            nonce,
        )

        valid_signature = verify_signature(
            item["public_key"],
            message,
            signature,
        )

        if not valid_signature:
            print(
            f"Invalid signature for "
            f"message from {item['username']}")
            continue

        decrypted_history.append({
            "username": item["username"],
            "content": message,
            "timestamp": item["timestamp"],
        })
    except Exception as error:
        # Ignore corrupted/tampered messages.
        print(
    f"SECURITY ALERT: Message from "
    f"{item['username']} failed "
    f"integrity/decryption check: {repr(error)}")
```

```
        continue

    await websocket.send(json.dumps({
        "type": "history",
        "messages": decrypted_history,
    }))

    users[websocket] = username

    print(f"{username} joined the chat.")

    await broadcast({
        "type": "system",
        "content": f"{username} joined the chat.",
    })

    try:
        async for raw_message in websocket:

            try:
                data = json.loads(raw_message)
            except json.JSONDecodeError:
                continue

            if data.get("type") != "chat":
                continue

            message = data.get("content", "").strip()
            signature_b64 = data.get("signature")

            if not message or not signature_b64:
                continue

            try:
                signature = base64.b64decode(signature_b64)
            except Exception:
                continue

            print(f"{username}: {message}")

            signature_valid = verify_signature(
                public_keys[username],
                message,
```

```
                signature,
                )

            if signature_valid:
                print(
                    f"[SIGNATURE VERIFIED] "
                    f"{username}: {message}"
                )
            else:
                print(
                    f"[SIGNATURE REJECTED] "
                    f"{username}: {message}"
                )

            if not signature_valid:
                await websocket.send(json.dumps({
                    "type": "error",
                    "message": "Invalid message signature.",
                }))
                continue

            ciphertext, nonce = encrypt_message(message)

            timestamp = datetime.now(
                timezone.utc
            ).isoformat()

            await asyncio.to_thread(
                store_message,
                username,
                public_keys[username],
                ciphertext,
                nonce,
                signature,
                timestamp
            )

            await broadcast({
                "type": "chat",
                "username": username,
                "content": message,
                "timestamp": timestamp,
            })
```

```
    except ConnectionClosed:
        pass

    finally:
        removed_username = await unregister_user(websocket)

        if removed_username is not None:
            print(f"{removed_username} left the chat.")
            await broadcast({
                "type": "system",
                "content": f"{removed_username} left the chat.",
            })


async def main():
    """Start the WebSocket server."""

    init_db()

    async with websockets.serve(
        handle_client,
        HOST,
        PORT,
        ssl=ssl_context,
    ):
        print(f"WebSocket server running on wss://{HOST}:{PORT}")
        print(f"Database: {DB_PATH}")
        print("Waiting for clients...")

        await asyncio.Future()


if __name__ == "__main__":
    asyncio.run(main())
```

App.jsx

```
import { useRef, useState } from "react";
import "./App.css";
```

```
const WS_URL = "wss://10.1.75.51:5266";

async function generateKeyPair() {
 return await window.crypto.subtle.generateKey(
   {
     name: "RSA-PSS",
     modulusLength: 2048,
     publicExponent: new Uint8Array([1, 0, 1]),
     hash: "SHA-256",
   },
   true,
   ["sign", "verify"]
 );
}

async function exportPublicKey(publicKey) {
 const spki = await window.crypto.subtle.exportKey(
   "spki",
   publicKey
 );

 return arrayBufferToBase64(spki);
}

async function signMessage(privateKey, message) {
 const data = new TextEncoder().encode(message);

 const signature = await window.crypto.subtle.sign(
   {
     name: "RSA-PSS",
     saltLength: 32,
   },
   privateKey,
   data
 );

 return arrayBufferToBase64(signature);
}

function arrayBufferToBase64(buffer) {
 const bytes = new Uint8Array(buffer);

 let binary = "";
```

```
 for (const byte of bytes) {
   binary += String.fromCharCode(byte);
 }

 return btoa(binary);
}

function formatTime(timestamp) {
 return new Date(timestamp).toLocaleTimeString([], {
   hour: "2-digit",
   minute: "2-digit",
 });
}

function App() {
 const [username, setUsername] = useState("");
 const [connected, setConnected] = useState(false);
 const [messages, setMessages] = useState([]);
 const [input, setInput] = useState("");

 const socketRef = useRef(null);
 const privateKeyRef = useRef(null);

 async function connect() {
   const name = username.trim();

   if (!name) {
     return;
   }

   // Generate signing key pair.
   const keyPair = await generateKeyPair();

   privateKeyRef.current = keyPair.privateKey;

   const publicKey = await exportPublicKey(
     keyPair.publicKey
   );

   const socket = new WebSocket(WS_URL);
   socketRef.current = socket;
```

```
  socket.onopen = () => {
    // Your backend expects the username as
    // the FIRST message after connection.
    socket.send(
      JSON.stringify({
        type: "register",
        username: name,
        public_key: publicKey,
      })
    );
  };

  socket.onmessage = (event) => {
      console.log("MESSAGE FROM SERVER:", event.data);
    const data = JSON.parse(event.data);

      console.log("PARSED DATA:", data);
  console.log("MESSAGE TYPE:", data.type);

    if (data.type === "error") {
      alert(data.message);
      socket.close();
      return;
    }

    if (data.type === "history") {
      {/*
      const history = payload.messages.map(
        (entry) =>
          `[${formatTime(entry.timestamp)}] ${entry.username}:
${entry.content}`,
      );
      */}

      setMessages(data.messages);
      setConnected(true);
      return;
    }

    if (data.type === "system") {

      setMessages(
        previous => [
```

```
          ...previous,
          {
            system: true,
            content: data.content,
          },
        ]
      );
    }

    if (data.type === "chat") {

      setMessages(
        previous => [
          ...previous,
          {
            username: data.username,
            content: data.content,
            timestamp: data.timestamp,
          },
        ]
      );

      return;
    }
  };


  socket.onclose = () => {
    setConnected(false);
      privateKeyRef.current = null;
  };

  socket.onerror = (error) => {
      console.error("WebSocket error:", error);
  };
}

async function sendMessage() {
  const message = input.trim();

  if (!message || !socketRef.current) {
    return;
  }
```

```
  if (socketRef.current.readyState !== WebSocket.OPEN) {
    return;
  }

  if (!privateKeyRef.current) {
    return;
  }

  // Sign the plaintext message.
  const signature = await signMessage(
    privateKeyRef.current,
    message
  );

  // const tamperedMessage = message + " TAMPERED";

  socketRef.current.send(
    JSON.stringify({
      type: "chat",
      // content: tamperedMessage,
    content: message,
      signature: signature,
    })
  );


  setInput("");

}

function handleKeyDown(event) {
  if (event.key === "Enter") {
    sendMessage();
  }
}

function disconnect() {
  socketRef.current?.close();
  socketRef.current = null;
  privateKeyRef.current = null;
  setMessages([]);
  setConnected(false);
```

```
}

if (!connected) {
  return (
    <div className="app">
      <div className="login-box">
        <h1>Group Chat</h1>

        <input
          type="text"
          placeholder="Enter username"
          value={username}
          onChange={(event) => setUsername(event.target.value)}
          onKeyDown={(event) => {
            if (event.key === "Enter") {
              connect();
            }
          }}
          maxLength={20}
        />

        <button onClick={connect}>
          Join Chat
        </button>
      </div>
    </div>
  );
}

return (
  <div className="app">
    <div className="chat-container">

      <header>
        <h1>Group Chat</h1>
        <span>Logged in as <b>{username}</b></span>

        <button onClick={disconnect}>
          Leave
        </button>
      </header>

      <div className="messages">
```

```
        {messages.map((message, index) => (
          <div className={
                message.system
                  ? "message system-message"
                  : "message"
              }
              key={index}>
            {message.system ? (
                  message.content
                  ) : (
                <>
                  <div>
                    <strong>{message.username}</strong>
                  </div>

                  <div>
                    {message.content}
                  </div>

                  <small>
                    {new Date(message.timestamp).toLocaleString()}
                  </small>
                </>
              )}
            </div>
        ))}
      </div>

      <div className="input-area">
        <input
          type="text"
          placeholder="Type a message..."
          value={input}
          onChange={(event) => setInput(event.target.value)}
          onKeyDown={handleKeyDown}
        />

        <button onClick={sendMessage}>
          Send
        </button>
      </div>

    </div>
```

```
    </div>
  );
}

export default App;
```

FEATURES ADDED:

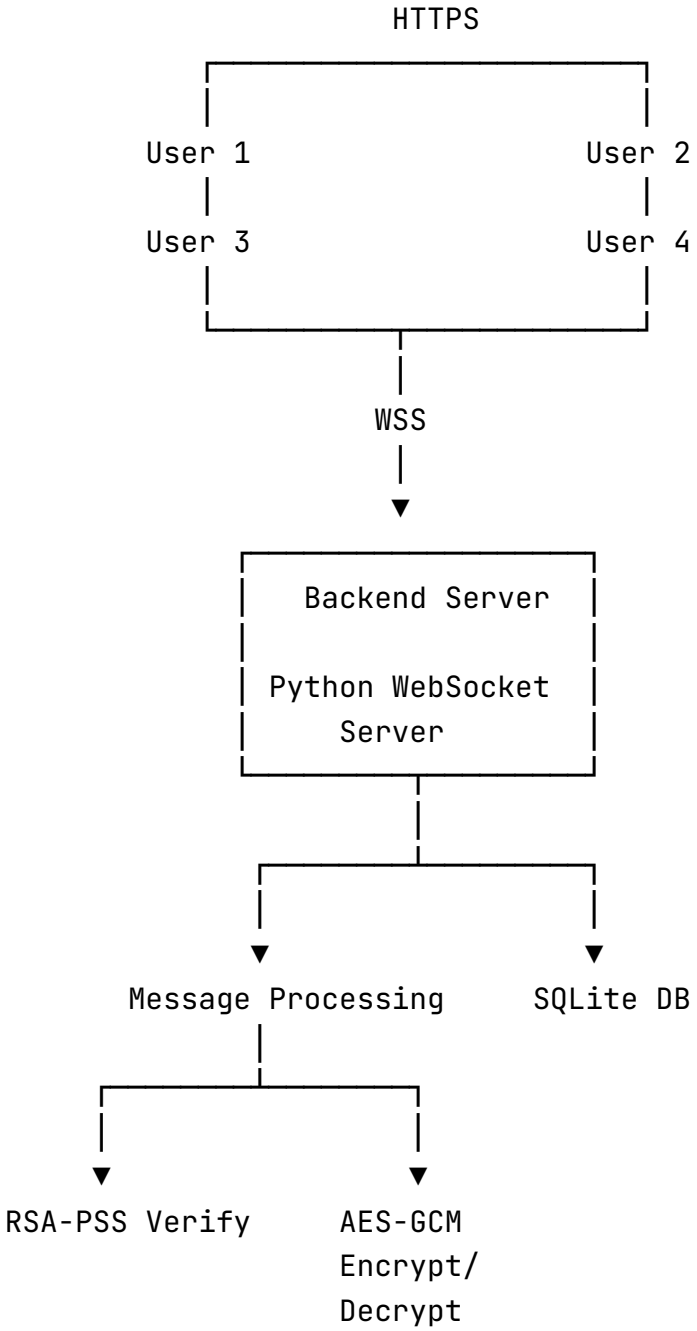
- Chat messages are encrypted using AES-GCM before being stored in the database.
- Each message is digitally signed using the sender's RSA private key.
- The server verifies message signatures before accepting messages.
- The public key used to sign each message is stored alongside that message.
- Historical messages are decrypted and signature-verified before being displayed.
- Tampering with stored ciphertext causes AES-GCM integrity verification to fail.
- Invalid or corrupted historical messages are rejected and not displayed.
- Chat history is stored persistently in SQLite.
- A new user receives the stored chat history when joining the room.
- HTTPS is used for the frontend.
- WSS (WebSocket Secure) is used for communication between the frontend and backend.

ARCHITECTURE

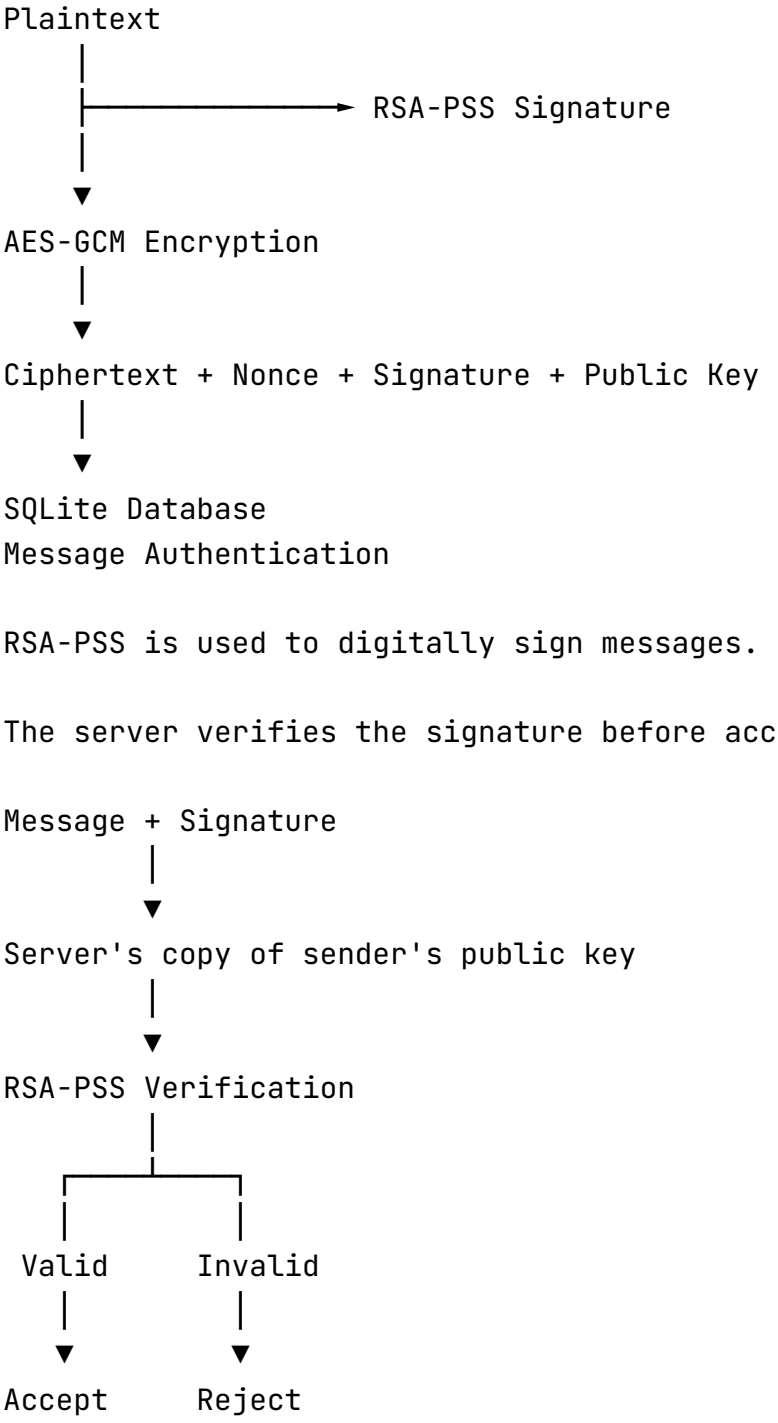
The application consists of:

- Backend: Python WebSocket server using the websockets library.
- Frontend: React application built with Vite.
- Database: SQLite database used to persist encrypted messages and their associated metadata.
- Communication: WebSockets provide persistent, bidirectional communication between frontend clients and the backend.

- Transport Security: HTTPS is used for the frontend and WSS is used for the WebSocket connection.
- Cryptography: AES-GCM provides message confidentiality and integrity, while RSA-PSS signatures provide message authentication.



CRYPTOGRAPHIC ARCHITECTURE



RSA-PSS is used to digitally sign messages.

The server verifies the signature before accepting a message:

When a user sends a message:

- The client signs the message using its RSA private key.
- The message and signature are sent to the backend over the WebSocket.

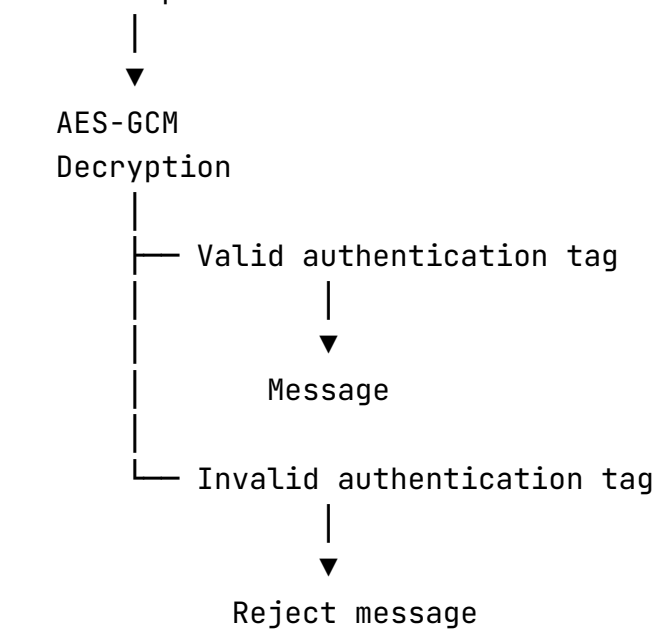
- The backend verifies the signature using the sender's public key.
- If the signature is invalid, the message is rejected.
- If the signature is valid, the message is encrypted using AES-GCM.
- The ciphertext, nonce, signature, sender username, public key, and timestamp are stored in SQLite.
- The message is broadcast to all currently connected users.

MESSAGE ENCRYPTION AND TAMPER DETECTION

Messages stored in SQLite are encrypted using AES-GCM.
The database stores:

- Username
- Public key
- Ciphertext
- Nonce
- Digital signature
- Timestamp
- AES-GCM provides both confidentiality and integrity.

If an attacker modifies the ciphertext stored in the database, the AES-GCM authentication check fails during decryption. The server then rejects the message rather than displaying corrupted or tampered content.



This allows tampering with the database to be demonstrated by modifying a stored ciphertext and reconnecting to the chat. The server detects the modification and reports an integrity/decryption failure.

CONTRIBUTION

1. Agastya Nath and Galaba Vamsi have worked on encryption.
2. Rahul Dev and Paritosh Lahre have worked on databases.

TUTORIAL VIDEO

demo_video.MP4